Computational Complexity (CPS-MIRI)
CP assignment
Report

Roger Canal Estrada, roger.canal@estudiantat.upc.edu
CPS git repository

Course 2025/2026

Variables

The model is constructed using three main types of integer variables, managed by the Gecode solver.

Decision Variables

- `open_streets_mat` (*IntVarArray of size* $n \times n$): This is the primary decision variable array, representing the final directed graph. Each element has a boolean domain $\{0, 1\}$. A value of 1 indicates that the street from crossing i to j is active and open to traffic, while 0 means it is closed.

Auxiliary Variables

- `travel_times` (*IntVarArray of size* $(n+1) \times n \times n$): Is a 3d matrix used to try all the possible combinations of new crossings from i->j. It represents the shortest travel time from i to j using intermediate vertices up to k. The domain of this variable is bounded from 0 to a large integer value (1000).

Objective Variable

- `converted_streets` (*IntVar*): The total number of two-way streets successfully converted to one-way. The domain is strictly between 0 and the total number of original two-way streets in the city.

Constraints

Base Case: (Layer 0 of the travel_times matrix)

For the first layer of `travel_times` ($k = 0$), the distances rely directly on the decisions made in `open_streets_mat`. An If-Then-Else (`ite`) constraint is used:

$$D_{i,j}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ T_{i,j} & \text{if } \texttt{open_streets_mat}_{i,j} = 1 \\ \infty & \text{otherwise} \end{cases} \tag{1}$$

Shortest Path Calculation

To compute the final travel times without cyclic assignments, non-binary constraints map the relationship between layers. For each layer $k \in \{0, \ldots, n-1\}$, the model enforces:

$$D_{i,j}^{(k+1)} = \min\left(D_{i,j}^{(k)}, \min(D_{i,k}^{(k)} + D_{k,j}^{(k)}, \infty)\right) \tag{2}$$

Time Contract Limit

Using the original shortest path $S_{i,j}$ and the tolerance $P\%$, a limit $L_{i,j} = S_{i,j} \times (1 + \frac{P}{100})$ is pre-computed. The constraint is applied to the final layer n of the `travel_times` matrix:

$$D_{i,j}^{(n)} \leq L_{i,j} \tag{3}$$

If the solver closes a street that forces a path to exceed this limit, immediately detects a domain wipeout and triggers a backtrack.

Street Integrity and Topology

The solver must respect the original infrastructure. For every pair of crossings (i, j):

- **Original Two-Way Streets:** At least one direction must remain open. With the constraint: $\texttt{open_streets_mat}_{i,j} + \texttt{open_streets_mat}_{j,i} \geq 1$. This prevents the solver from "deleting" streets entirely.
- **Original One-Way Streets:** Forced to remain exactly as they were (e.g., $i \to j = 1$ and $j \to i = 0$).
- **Non-existent Streets:** Forced to 0 in both directions.

Objective Calculation

The objective value is the total number of two-way streets converted to one-way. This can be computed as the sum of all active directions. Since an unconverted street contributes 2 and a converted street contributes 1:

$$\text{converted_streets} = (2 \times \text{total_two_way}) - \text{Sum}(\texttt{active_two_way_edges}) \tag{4}$$

Some notes

Branch and Bound (BAB): I used the BAB algorithm to search in the tree solutions, plus with the constrain method, it ensures that the next solution will have a higher `converted_streets` value.

Branching: In the branch method, it branches in `open_streets_mat` using `INT_VAR_NONE()` and `INT_VAL_MIN()`. By attempting to assign 0 first, the solver greedily tries to close streets (convert them) early in the search tree.

Code notes: In the sdp.cpp file, I added comments on each step. This report does an overall explanation of thecnical concepts, but I hope that the code comments helps to understand my implementation!